SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 5

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 5: Functions and decisions

Reusable code, if and else

Yesterday you wrote instructions that run once, from top to bottom. Today you package them into **functions**: named pieces of code you can run again and again, with different numbers each time.

The page shows four show cards. Every card has grey notes like *(write watchTime)*. A helper file, **page.js**, is already written: it runs after your code and uses your functions to fill in the cards. So each function you finish makes all four cards light up at once.

You'll also learn the most important difference in today's lesson: a function can *print* a value, or *give it back*. Only one of them fills the cards.

Step 1. Your first function ~15 min

Teaches: `function name() { ... }` defines a function; `name();` calls it.

TASK

1. Define a function called `sayHello` that logs **Hello from a function!**
2. Under it, call it twice.

COMMON MISTAKES

- Defining the function but never calling it (nothing appears).
- Calling it without brackets: `sayHello;` does nothing.
- Putting the calls inside the function's own `{ }`.

A CORRECT ANSWER

```
// app.js: your functions. page.js uses them to fill in the cards.

function sayHello() {
  console.log("Hello from a function!");
}

sayHello();
sayHello();
```

Step 2. Parameters and arguments ~15 min

Teaches: Parameters are named in the definition; arguments are the values in the call.

TASK

1. Define a function `greet` with one parameter, `name`. It logs a greeting with the name, like **Hello, Ana!**
2. Call it twice, with two different names.

COMMON MISTAKES

- Quotes around the parameter inside the function: `"Hello, name!"`.
- Calling with no argument: `greet()` prints *Hello, undefined!*

NOTE

If a student asks about `undefined`: it's JavaScript's word for "no value yet". It becomes important in the next two steps.

A CORRECT ANSWER

```
// ... unchanged ...
sayHello();
sayHello();

function greet(name) {
  console.log(`Hello, ${name}!`);
}

greet("Ana");
greet("Ben");
```

Step 3. Two parameters ~15 min

Teaches: Several parameters are separated by commas; arguments go in the same order.

TASK

1. Define `watchTime` with two parameters, `episodes` and `runtime`. It logs the hours: episodes times runtime, divided by 60.
2. Call it for Friends, `watchTime(236, 30)`, and for Breaking Bad, `watchTime(62, 60)`.

Then look closely:

- The Console shows more numbers than you asked for. Where do the others come from?
- The cards say *Watch it all: undefined hours*. Why? The next step explains.

COMMON MISTAKES

- Arguments in the wrong order (the result is the same here, but not in general).
- Using the numbers inside the function instead of the parameters: `console.log(236 * 30 / 60)`.

A CORRECT ANSWER

```
// ... unchanged ...
greet("Ana");
greet("Ben");

function watchTime(episodes, runtime) {
  console.log(episodes * runtime / 60);
}

watchTime(236, 30);
watchTime(62, 60);
```

Step 4. Give the answer back with return ~15 min

Teaches: `return` gives a value back to the caller; `console.log` only shows it to you.

TASK

1. Change `watchTime` so that it **returns** the hours, rounded with `Math.round`, instead of logging them.
2. Your two calls now print nothing. Replace them with one line that logs the answer: `console.log(watchTime(62, 60));`.

Look at the cards: the watch times appear.

COMMON MISTAKES

- Keeping `console.log` inside and adding `return` after it (fine, but remove the log).
- `return` on its own line with the value on the next line: JavaScript returns `undefined`.
- Writing lines after `return` and wondering why they never run.

NOTE

This is the key idea of the day. If a student is unsure, ask: "Who needs the answer, you or page.js?"

A CORRECT ANSWER

```
// ... unchanged ...

function watchTime(episodes, runtime) {
  return Math.round(episodes * runtime / 60);
}

console.log(watchTime(62, 60));
```

Step 5. Decisions inside a function ~20 min

Teaches: Functions can use `if / else if / else` and return a different value in each block.

TASK

Write `ratingLabel(rating)` that returns:

- **Must watch** for 9 or more
- **Good** for 8 or more
- **OK** for 6 or more
- **Skip** for anything lower

Test it with `console.log(ratingLabel(6.5));` and `console.log(ratingLabel(3));`. All our shows are rated 8 or more, so only the Console shows the last two labels.

COMMON MISTAKES

- The conditions in the wrong order (`>= 6` first catches everything).
- `>` instead of `>=`: a rating of exactly 9 becomes Good.
- `console.log` in each block instead of `return`.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(watchTime(62, 60));

function ratingLabel(rating) {
  if (rating >= 9) {
    return "Must watch";
  } else if (rating >= 8) {
    return "Good";
  } else if (rating >= 6) {
    return "OK";
  } else {
    return "Skip";
  }
}

console.log(ratingLabel(6.5));
console.log(ratingLabel(3));
```

Step 6. Compare two parameters ~15 min

Teaches: Conditions can compare parameters with each other: `if (premiered === ended)`.

TASK

Write `yearsLabel(premiered, ended)`:

- when both years are the same, return only one year, like **2019**
- otherwise, return both with a dash between them, like **1994–2004**

The long dash `–` can be hard to type: a normal `-` is fine too.

COMMON MISTAKES

- One `=` in the condition: it changes `premiered` and the result is always the second year.
- Returning `"premiered–ended"` in quotes instead of a template literal.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(ratingLabel(6.5));
console.log(ratingLabel(3));

function yearsLabel(premiered, ended) {
  if (premiered === ended) {
    return `${premiered}`;
  } else {
    return `${premiered}–${ended}`;
  }
}
```

Step 7. Hours and minutes ~20 min

Teaches: `Math.floor()` rounds down; `%` is the remainder of a division.

TASK

In `formatRuntime`, replace the three `TODO` comments with an `if / else if / else` that returns:

1. `"45 min"` style when `hours` is 0
2. `"1 h"` style when `rest` is 0
3. `"1 h 30 min"` style otherwise

Use template literals with `${hours}` and `${rest}`. Test it: `console.log(formatRuntime(90));`.

COMMON MISTAKES

- Checking `minutes === 0` instead of `hours === 0`.
- Forgetting `return` in one of the blocks.

NOTE

If `%` is confusing, use a clock: 90 minutes is 1 hour and 30 left over. `90 % 60` gives the left-over part.

A CORRECT ANSWER

```
// ... unchanged ...
  }
}

function formatRuntime(minutes) {
  const hours = Math.floor(minutes / 60);
  const rest = minutes % 60;
  if (hours === 0) {
    return `${rest} min`;
  } else if (rest === 0) {
    return `${hours} h`;
  } else {
    return `${hours} h ${rest} min`;
  }
}

console.log(formatRuntime(90));
```

Step 8. Functions that use functions ~15 min

Teaches: Functions can call other functions, and can return `true` or `false`.

TASK

Write `isQuickWatch(episodes, runtime)`. It returns `true` when the whole show takes **10 hours or less**, and `false` otherwise.

Use your `watchTime` function inside it. Only one of the four shows is a quick watch. Can you guess which one before you look?

COMMON MISTAKES

- Returning the strings `"true"` / `"false"` instead of booleans.
- Copying the maths from `watchTime` instead of calling it.
- `< 10` instead of `<= 10`.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(formatRuntime(90));

function isQuickWatch(episodes, runtime) {
  return watchTime(episodes, runtime) <= 10;
}
```

Step 9. Arrow functions ~15 min

Teaches: Arrow functions: `const name = (a, b) => value;`

TASK

Rewrite `watchTime` as an arrow function, on one line:

1. Write `const watchTime = (episodes, runtime) => ...` with the calculation (and `Math.round`) after the arrow.
2. Delete the old `function watchTime ...` block, from `function` to its `}`. Two functions with the same name cause an error.

The cards and the Console should look exactly the same as before.

COMMON MISTAKES

- Keeping both versions: "Identifier 'watchTime' has already been declared".
- `{ }` around the body without `return`: the function returns `undefined`.
- Putting the arrow function below the line that calls it.

A CORRECT ANSWER

```
// ... unchanged ...
greet("Ben");

const watchTime = (episodes, runtime) => Math.round(episodes * runtime / 60);

console.log(watchTime(62, 60));
// ... unchanged ...
```

Step 10. A default value ~20 min

Teaches: Default values: `function f(a, b = "x")` uses `"x"` when `b` is left out.

TASK

Write `plural(count, word = "episode")`:

- when `count` is 1, return the count and the word: **1 episode**
- otherwise, return the count, the word and an **s**: **236 episodes**

page.js calls `plural(episodes)` with one argument, so the default is used. Test the other word in the Console: `console.log(plural(3, "season"));` should print **3 seasons**.

COMMON MISTAKES

- `word == "episode"` or `word: "episode"` in the brackets instead of `word = "episode"`.
- Writing `episodes` in the text instead of `${word}s`.

NOTE

All six grey notes should now be filled. Students who finish early can try the extra challenges, which add two more lines to every card.

A CORRECT ANSWER

```
// ... unchanged ...
  return watchTime(episodes, runtime) <= 10;
}

function plural(count, word = "episode") {
  if (count === 1) {
    return `${count} ${word}`;
  } else {
    return `${count} ${word}s`;
  }
}

console.log(plural(3, "season"));
```

Extra 1. Episode codes like S01E05 ~15 min

Teaches: `String(n).padStart(2, "0")` gives two-digit numbers like 05.

TASK

Write `episodeCode(season, episode)` that returns a code like **S01E05**. Each card gets a new line: *First episode: S01E01*.

Test it: `console.log(episodeCode(10, 18));` should print **S10E18**.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(plural(3, "season"));

function episodeCode(season, episode) {
  const s = String(season).padStart(2, "0");
  const e = String(episode).padStart(2, "0");
  return `S${s}E${e}`;
}

console.log(episodeCode(10, 18));
```

Extra 2. How many days to binge it? ~15 min

Teaches: `Math.ceil()` rounds up to the next whole number.

TASK

Write `binge(hours, hoursPerDay)` that returns how many days it takes to watch `hours` of TV at `hoursPerDay` a day. A started day counts as a whole day, so round up.

Every card gets a new line: *At 2 hours a day: 59 days*.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(plural(3, "season"));

function binge(hours, hoursPerDay) {
  return Math.ceil(hours / hoursPerDay);
}
```